Week 9 Meeting 1: Quantum Chemistry with Qiskit Nature

Student Lab Handout

Session Overview

Topic: Ground-state energy of H_2 using VQE
Duration: 1h15

Session Objectives

- Explain the basic quantum chemistry → qubit mapping workflow
- Build a molecular Hamiltonian using Qiskit Nature
- Run a Variational Quantum Eigensolver (VQE) on a simulator
- Interpret the ground-state energy result
- Understand what part would run on a real quantum computer

Timed Agenda (75 minutes)

1. **0:00-0:10**: Conceptual introduction
2. **0:10-0:25**: Molecule & Hamiltonian construction
3. **0:25-0:45**: Qubit mapping & VQE
4. **0:45-1:00**: Running & analyzing results
5. **1:00-1:15**: Extensions + discussion

Setup

```
#%pip install qiskit qiskit-nature
#%pip install --upgrade qiskit-nature qiskit-algorithms
#%pip install pyscf
```

```
import matplotlib.pyplot as plt
import numpy as np
```

1 Introduction: Motivation and Overview

Quantum chemistry aims to predict the properties of molecules starting from the laws of quantum mechanics. One of the most fundamental quantities of interest is the *ground-state energy*, i.e. the lowest possible energy that a molecule can have. This energy determines many physical and chemical properties such as molecular stability, reaction rates, and bond strengths.

1.1 Why is Quantum Chemistry Hard?

Exponential scaling on classical computers. Molecules are made of electrons, and each electron can occupy many possible quantum states. When we try to describe several interacting electrons exactly, the size of the mathematical space required grows *exponentially* with the number of electrons. As a result, the memory and computation time needed on a classical computer quickly become unmanageable, even for relatively small molecules. This is why exact simulations of quantum chemistry problems are typically limited to very small systems.

Electrons are quantum objects. Electrons do not behave like classical particles. They exhibit uniquely quantum phenomena such as *superposition* and *entanglement*, and they are described by wavefunctions rather than definite trajectories. Because of this, accurately modeling electronic structure requires solving quantum mechanical equations, such as the Schrödinger equation. Quantum computers are naturally suited to represent and process such quantum states, since qubits themselves obey the laws of quantum mechanics.

1.2 Our Goal

The main goal in this lab is to compute the *ground-state energy* of a molecule. Mathematically, this corresponds to finding the smallest eigenvalue of the molecular Hamiltonian, which is an operator encoding all kinetic and interaction energies of the electrons and nuclei. Rather than solving this problem exactly, we use a hybrid quantum-classical algorithm that can run on today's noisy quantum hardware.

1.3 From Molecules to Quantum Circuits

In practice, several conceptual steps are required to go from chemistry to a quantum computation. The workflow used in this lab is:

$$\text{Molecule} \longrightarrow \text{Fermions} \longrightarrow \text{Qubits} \longrightarrow \text{VQE} \longrightarrow \text{Energy}.$$

- Starting from a classical description of a molecule, we construct a quantum-mechanical Hamiltonian describing interacting electrons (fermions).
- Fermionic operators are then mapped onto qubit operators, which can be implemented on a quantum computer.

- A variational quantum algorithm, called the *Variational Quantum Eigensolver* (VQE), is used to approximate the ground-state energy.
- The output of the algorithm is an estimate of the molecular ground-state energy.

1.4 Why the Hydrogen Molecule?

Here, we focus on the hydrogen molecule, H_2, for several important reasons:

- It is the simplest molecule that exhibits electronic bonding.
- It can be represented using only four qubits, making it feasible for near-term quantum devices.
- Despite its simplicity, it captures all essential aspects of a genuine quantum chemistry problem, including electron-electron interactions.

For these reasons, H_2 serves as an ideal testbed for learning how quantum computers can be applied to quantum chemistry.

2 Defining the Molecular System with Qiskit Nature

Before a quantum computer can be used to solve a chemistry problem, we must first translate the molecular system into a form that can be processed algorithmically. In this lab, we use `Qiskit Nature`, which provides high-level tools to interface with established classical quantum chemistry software.

2.1 Using a Classical Chemistry Engine

The first step is to import a classical electronic structure solver. Here, we use `PySCF`, a widely used quantum chemistry package that computes molecular integrals and electronic Hamiltonians.

```
from qiskit_nature.second_q.drivers import PySCFDriver
from qiskit_nature.units import DistanceUnit
```

The `PySCFDriver` acts as a bridge between traditional quantum chemistry methods and quantum computing workflows. It allows us to define a molecule using familiar chemical concepts such as atomic positions and basis sets. The `DistanceUnit` class ensures that all spatial coordinates are interpreted consistently.

2.2 Step 1: Setting Up the Molecule

We now specify the molecular system we want to study; the molecule is hydrogen (H_2), consisting of two hydrogen atoms separated by a fixed distance.

```
driver = PySCFDriver(
    atom="H 0 0 0; H 0 0 0.735",
    basis="sto3g",
    unit=DistanceUnit.ANGSTROM
)
```

Each part of this definition has a clear physical meaning:

- The `atom` string specifies the type and position of each atom in three-dimensional space. Here, one hydrogen atom is placed at the origin, and the second is placed 0.735 Å away along the z-axis. This distance corresponds approximately to the equilibrium bond length of H_2.

- The `basis` parameter selects the basis set used to represent the electronic wavefunctions. The `sto3g` basis is a minimal basis set, meaning it uses the smallest number of functions necessary to describe each atomic orbital. While not highly accurate, it is ideal for educational purposes and small quantum simulations.
- The `unit` parameter specifies that distances are given in Ångströms, a standard unit in molecular physics and chemistry.

2.2.1 The STO-3G Basis Set

In quantum chemistry calculations, the electronic structure of a molecule is described by electronic wavefunctions, also called *orbitals*. Because solving the electronic Schrödinger equation exactly is not feasible for most systems, these orbitals are approximated using a predefined set of mathematical functions known as a *basis set*.

What Is a Basis Set? A basis set is a collection of functions used to represent electronic wavefunctions. Instead of solving for the exact wavefunction, we express it as a linear combination of simpler functions. The choice of basis set determines the balance between:

- computational cost, and
- accuracy of the electronic structure calculation.

Using more basis functions generally leads to higher accuracy, but also increases the computational resources required.

Slater-Type Orbitals (STOs) From a physical standpoint, electrons in atoms are well described by *Slater-type orbitals* (STOs), which have the general form:

$$\psi(r) \propto e^{-\zeta r},$$

where r is the distance from the nucleus and ζ is a decay parameter.
These orbitals:

- have the correct exponential decay behavior,
- describe electron density near the nucleus accurately.

However, STOs are computationally expensive because integrals involving these functions are difficult to evaluate efficiently on classical computers.

Gaussian-Type Orbitals (GTOs) To reduce computational cost, quantum chemistry calculations often use *Gaussian-type orbitals* (GTOs), which take the form:

$$\psi(r) \propto e^{-\alpha r^2}.$$

Gaussian functions do not perfectly reproduce the physical behavior of atomic orbitals, but they offer a key advantage: integrals involving Gaussian functions can be computed efficiently and analytically.

For this reason, most practical quantum chemistry calculations use Gaussian functions as building blocks.

The Meaning of STO-3G The STO-3G basis set combines the physical intuition of Slater-type orbitals with the computational efficiency of Gaussian functions.

- **STO** indicates that the target shape is a Slater-type orbital.
- **3G** means that each Slater-type orbital is approximated by a linear combination of *three* Gaussian functions.

In other words, each atomic orbital is represented as:

$$\text{STO} \approx \text{Gaussian}_1 + \text{Gaussian}_2 + \text{Gaussian}_3.$$

This approximation significantly reduces computational cost while retaining a qualitatively reasonable description of the electron distribution.

Limitations It is important to note that STO-3G is not intended for high-accuracy chemical predictions. More advanced basis sets (such as 6-31G or correlation-consistent basis sets) are required for quantitative agreement with experiment. However, these larger basis sets would currently require more qubits than are available on today's quantum hardware.

2.3 Step 2: Running the Classical Calculation

Once the molecule has been defined, we run the classical computation:

```
problem = driver.run()
```

This command performs a series of classical quantum chemistry calculations, including:

- computing molecular orbitals,
- determining the number of electrons,
- constructing the second-quantized electronic Hamiltonian.

The result is stored in a `problem` object. This object contains all the chemical information required for the subsequent quantum computation but does not yet involve any qubits.

2.4 Step 3: Inspecting the System

To better understand the system we have defined, we can inspect some of its basic properties:

```
# --- Molecular Data Dashboard ---
# This cell extracts all the key physical and chemical properties stored in 'problem'

print("=== H2 MOLECULE SUMMARY ===")

# 1. Basic Electronic Structure
# num_particles returns (alpha_electrons, beta_electrons)
alpha_e, beta_e = problem.num_particles
print(f"Total Electrons:        {sum(problem.num_particles)} ({alpha_e} up, {beta_e} down)")
print(f"Spatial Orbitals:       {problem.num_spatial_orbitals} (Physical 'slots' for electrons)")
print(f"Spin Orbitals:          {problem.num_spin_orbitals} (Total quantum states available)")

# 2. Energy Reference Points (In Hartree units)
# The Hartree-Fock energy is the best 'classical' guess for the ground state
print(f"\n--- Energy Benchmarks (Hartree) ---")
```

```
print(f"Nuclear Repulsion:      {problem.nuclear_repulsion_energy:.5f}")
print(f"Hartree-Fock Energy:    {problem.reference_energy:.5f}")

# 3. Orbital Energies
# These are the energy levels of the individual molecular orbitals
print(f"\n--- Orbital Energy Levels ---")
for i, energy in enumerate(problem.orbital_energies):
    status = "Occupied" if i < alpha_e else "Empty"
    print(f"Orbital {i}: {energy:10.5f} Ha [{status}]")

# 4. Symmetry & Geometry
print(f"\n--- Geometry Info ---")
# The 'molecule' attribute contains the raw coordinates and atoms
mol = problem.molecule
print(f"Atoms: {mol.symbols}")
# Calculate bond length (distance between the two H atoms)
coords = np.array(mol.coords)
dist = np.linalg.norm(coords[0] - coords[1])
print(f"Bond Length: {dist:.4f} Bohr")
# 1 \AA ~ 1.8897 Bohr
print("============================")
```

```
import matplotlib.pyplot as plt
import numpy as np

# 1. Setup the 4 spin-orbital energies
# In H2 (without a magnetic field), Alpha and Beta energies are identical (degenerate)
spatial_energies = problem.orbital_energies
# We create a list of 4 energies: [Alpha_Bonding, Beta_Bonding, Alpha_Anti, Beta_Anti]
spin_energies = [spatial_energies[0], spatial_energies[0], spatial_energies[1], spatial_energies[1]]
spin_labels = [r'Bonding $\uparrow$', r'Bonding $\downarrow$', r'Anti-bond $\uparrow$', r'Anti-bond $\downarrow$']

# 2. Setup the Plot
plt.figure(figsize=(8, 6))
x_positions = [0.2, 0.4, 0.7, 0.9] # Space them out to see 4 distinct lines

for i in range(4):
    # Draw a shorter horizontal line for each spin-orbital
    plt.hlines(spin_energies[i], x_positions[i]-0.08, x_positions[i]+0.08, lw=3, color='navy')
    plt.text(x_positions[i], spin_energies[i] - 0.15, spin_labels[i],
             ha='center', fontsize=10, fontweight='bold')

# 3. Occupy the orbitals (H2 has 2 electrons)
# Electron 1: Alpha Bonding
plt.annotate('', xy=(x_positions[0], spin_energies[0] + 0.2),
             xytext=(x_positions[0], spin_energies[0] - 0.2),
             arrowprops=dict(arrowstyle='->', lw=2.5, color='red', mutation_scale=20))

# Electron 2: Beta Bonding
plt.annotate('', xy=(x_positions[1], spin_energies[1] - 0.2),
             xytext=(x_positions[1], spin_energies[1] + 0.2),
             arrowprops=dict(arrowstyle='->', lw=2.5, color='green', mutation_scale=20))

# 4. Formatting
plt.title(r"H$_2$ Spin-Orbital Diagram (4 States)", fontsize=16)
plt.ylabel("Energy (Hartree)", fontsize=14)
```

```
plt.xticks([0.3, 0.8], ["Bonding Sector", "Anti-bonding Sector"], fontsize=12)
plt.xlim(0, 1.1)
plt.ylim(min(spin_energies)-0.8, max(spin_energies)+0.8)
plt.grid(axis='y', linestyle='--', alpha=0.3)

plt.show()
```

3 Map Fermions → Qubits

Quantum computers operate on *qubits*, not directly on physical particles such as electrons. A qubit is an abstract two-level quantum system, mathematically described by a two-dimensional Hilbert space. While real quantum hardware may use physical systems such as superconducting circuits, trapped ions, or photons, these systems do not directly represent individual electrons in a molecule.

In contrast, quantum chemistry problems are naturally formulated in terms of *electrons*, which are quantum particles obeying the Pauli exclusion principle. Electrons are examples of *fermions*, meaning that their quantum states must be antisymmetric under particle exchange. Because qubits do not inherently obey fermionic statistics, we cannot directly simulate electrons using qubits without an additional transformation.

Therefore, an intermediate step is required to rewrite the electronic structure problem into a form that can be implemented on a quantum computer.

3.1 Fermions and Second Quantization

In quantum chemistry, electronic systems are commonly described using *second quantization*. In this framework:

- electrons occupy molecular orbitals,
- each orbital can be either occupied or unoccupied,
- creation and annihilation operators are used to add or remove electrons from orbitals.

Fermionic Systems Electrons are fermions, which means they obey Fermi-Dirac statistics and the Pauli exclusion principle: no two electrons may occupy the same quantum state. In molecular systems such as H_2, electrons occupy *spin-orbitals*, each of which can be either unoccupied or occupied by a single electron. This "occupation-number" description makes fermionic systems naturally suited to the language of *second quantization*.

Second Quantization for H_2 In second quantization, we describe the electronic structure of the molecule not in terms of wavefunctions depending on continuous coordinates, but in terms of operators acting on discrete spin-orbitals. For H_2 in a minimal basis, there are four relevant spin-orbitals:

- bonding orbital with spin up ($\uparrow$),
- bonding orbital with spin down ($\downarrow$),
- antibonding orbital with spin up ($\uparrow$),
- antibonding orbital with spin down ($\downarrow$).

Creation operators $a_p^\dagger$ (written in Qiskit output as `p+`) add an electron to spin-orbital p, and annihilation operators a_p (written in Qiskit as `p`) remove an electron from spin-orbital p. These operators automatically enforce fermionic statistics via their anticommutation relations.

Running

```
fermionic_hamiltonian = hamiltonian.second_q_op()
```

produces the electronic Hamiltonian of the H_2 molecule written in *second-quantized form*. In this formalism, the Hamiltonian is expressed in terms of fermionic creation and annihilation operators acting on spin-orbitals:

$$\hat{H} = \sum_{pq} h_{pq}\, a_p^\dagger a_q \;+\; \frac{1}{2} \sum_{pqrs} h_{pqrs}\, a_p^\dagger a_q^\dagger a_s a_r.$$

Here:

- $a_p^\dagger$ creates an electron in spin-orbital p,
- a_p annihilates an electron in spin-orbital p,
- h_{pq} are the one-electron integrals,
- h_{pqrs} are the two-electron integrals.

The Hamiltonian naturally separates into two parts.

One-body term. The first sum,

$$\sum_{pq} h_{pq}\, a_p^\dagger a_q,$$

describes single-electron physics:

- kinetic energy of the electrons,
- attraction of the electrons to the nuclei,
- effective energies of the molecular orbitals.

Terms of this form correspond to electrons occupying orbitals or moving between orbitals.

Two-body term. The second sum,

$$\frac{1}{2} \sum_{pqrs} h_{pqrs}\, a_p^\dagger a_q^\dagger a_s a_r,$$

encodes electron-electron interactions. These terms describe how the presence of one electron affects the energy and motion of another electron, and are responsible for electron correlation effects beyond mean-field theory.

Connection to the Python Output Each operator string printed in the Python output is one term from the sums above. For example:

- a term like $0^+\,0$ corresponds to $a_0^\dagger a_0$, a one-body term measuring the occupation energy of spin-orbital 0. It represents the energy of an electron occupying the bonding spin-up spin-orbital (index 0). More generally, $a_0^\dagger a_0$ counts the number of electrons in spin-orbital 0.

- a term like $0^+\,1^+\,3\,2$ corresponds to $a_0^\dagger a_1^\dagger a_3 a_2$, a two-body term describing electron-electron repulsion. It corresponds to two electrons interacting and exchanging orbitals. Its interpretation is:

  1. Remove an electron from spin-orbital 2 (antibonding ↑),
  2. Remove an electron from spin-orbital 3 (antibonding ↓),
  3. Create an electron in spin-orbital 1 (bonding ↓),
  4. Create an electron in spin-orbital 0 (bonding ↑).

  Such terms encode the electron-electron repulsion that couples different electronic configurations.

3.2 Mapping Fermions to Qubits

To run a quantum chemistry calculation on a quantum computer, we must translate fermionic operators into operators acting on qubits. This process is known as *fermion-to-qubit mapping*.

Schematically, the workflow is:

$$\text{Fermions} \longmapsto \text{Qubits}.$$

Several such mappings exist, and in this lab we use the *Jordan-Wigner mapping*, which is conceptually the simplest and most intuitive.

In a 4-qubit simulation using the Jordan-Wigner mapping, each qubit corresponds to one spin-orbital. For the H_2 molecule in a minimal basis, these spin-orbitals are:

- bonding orbital with spin up and down,
- antibonding orbital with spin up and down.

Each of these spin-orbitals is mapped one-to-one onto a qubit:

$$1 \text{ spin-orbital} \longleftrightarrow 1 \text{ qubit}.$$

A convenient labeling is:

- Qubit 0: Bonding orbital, spin up (↑)
- Qubit 1: Bonding orbital, spin down (↓)
- Qubit 2: Antibonding orbital, spin up (↑)
- Qubit 3: Antibonding orbital, spin down (↓)

NB: software interleaves the spins: 0 (↑), 1 (↓), 2 (↑), 3 (↓)

Occupation and Initial State In the ground state of H_2:

- both electrons occupy the bonding orbital,
- one electron has spin up and the other has spin down,
- the antibonding orbitals are unoccupied.

This corresponds to:

- occupied bonding spin-orbitals → qubits in state $|1\rangle$,
- empty antibonding spin-orbitals → qubits in state $|0\rangle$.

Therefore, the initial state of the quantum computer is $|1100\rangle$.
The qubit string is ordered as:

$$|\text{Bonding } \uparrow, \text{ Bonding } \downarrow, \text{ Antibonding } \uparrow, \text{ Antibonding } \downarrow\rangle\,.$$

Physical Interpretation

- $|1\rangle$ means "this orbital is occupied by an electron",
- $|0\rangle$ means "this orbital is empty",
- the bitstring $|1100\rangle$ represents the Hartree-Fock reference state,
- this state is typically used as the starting point for the Variational Quantum Eigensolver (VQE).

Fermionic creation and annihilation operators are represented using combinations of Pauli operators acting on these qubits. Additional strings of σ_z operators are introduced to ensure that the correct fermionic anticommutation relations are preserved. While the full mathematical expressions can be intricate, the essential idea is straightforward:

> Orbital occupation is encoded directly into qubit states, with extra operators enforcing fermionic behavior.

```python
# Import the tool that translates chemistry into quantum bits (qubits)
from qiskit_nature.second_q.mappers import JordanWignerMapper

# 1. Choose a Translation Strategy:
# Computers use qubits (0 and 1), but chemistry uses fermions (electrons).
# The JordanWignerMapper is the standard 'dictionary' used to translate
# electron behaviors into quantum gate instructions.
mapper = JordanWignerMapper()

# 2. Grab the Chemistry Math:
# We pull the 'Hamiltonian' (the mathematical description of the
# molecule's energy) from the 'problem' object we created earlier.
hamiltonian = problem.hamiltonian

# 3. Translate to Qubits:
# We take the complex second-quantized (chemical) math and 'map' it
# into a qubit operator (a list of Pauli strings like XIIZ, ZIZI, etc.).
# This is what the quantum computer actually runs.
qubit_op = mapper.map(hamiltonian.second_q_op())

# 4. Check the result:
print(f"Number of qubits needed: {qubit_op.num_qubits}")
```

The Jordan-Wigner Transformation The Jordan-Wigner (JW) mapping provides a direct correspondence between fermionic occupation numbers and qubit states. Because fermions must satisfy anti-commutation relations, the JW mapping introduces a "string" of Z operators to maintain the correct phase.

The JW mapping rules for an operator at site j are:

$$\hat{a}_j \to \frac{1}{2}(X_j + iY_j) \otimes Z_{j-1} \otimes \cdots \otimes Z_0 \tag{1}$$

$$\hat{a}_j^\dagger \to \frac{1}{2}(X_j - iY_j) \otimes Z_{j-1} \otimes \cdots \otimes Z_0 \tag{2}$$

$$\hat{a}_j^\dagger \hat{a}_j \to \frac{1}{2}(I_j - Z_j) \tag{3}$$

Application to H_2 Simulation Using the STO-3G basis, H_2 is represented by 4 spin-orbitals (Bonding $\uparrow, \downarrow$ and Anti-bonding $\uparrow, \downarrow$). Under the Jordan-Wigner mapping, these 4 spin-orbitals map to 4 qubits.

A single fermionic term such as $-1.256\hat{a}_0^\dagger\hat{a}_0$ is translated into a Pauli string:

$$-1.256\hat{a}_0^\dagger\hat{a}_0 \xrightarrow{JW} -0.628(I_0 - Z_0) \tag{4}$$

```python
# --- Qubit Hamiltonian Dashboard ---
print("=== QUBIT OPERATOR (MACHINE CODE) SUMMARY ===")

# 1. Basic Stats
num_qubits = qubit_op.num_qubits
num_terms = len(qubit_op)
print(f"Number of Qubits:      {num_qubits}")
print(f"Total Pauli Terms:     {num_terms}")
print("-" * 45)

# 2. Breaking down the Pauli Strings
# We sort them by the absolute value of the coefficient to see the most 'important' terms first
sorted_terms = sorted(qubit_op.to_list(), key=lambda x: abs(x[1]), reverse=True)

print(f"{'Pauli String':<15} | {'Coefficient (Energy Contribution)':<30}")
print("-" * 45)

# Display the first 10 terms (the most significant ones)
for pauli_str, coeff in sorted_terms[:10]:
    # coeff is often a complex number, we take the real part for energy
    real_coeff = coeff.real
    print(f"{pauli_str:<15} | {real_coeff:>+10.5f} Ha")

if num_terms > 10:
    print(f"... and {num_terms - 10} more terms.")

# 3. Physical Interpretation for Beginners
print("-" * 45)
print("INTERPRETATION:")
print(f"- 'IIII' is a constant energy offset.")
print(f"- 'Z' terms (like ZIII) represent orbital energy (On-site).")
print(f"- 'ZZ' terms (like ZZII) represent electron repulsion (Coulomb).")
print(f"- 'XXXX' or 'YYYY' terms represent electron hopping/exchange.")
print("=============================================")
```

By summing all translated terms, the total Hamiltonian is represented as a linear combination of Pauli strings (e.g., $IIII, ZIII, ZZII, \ldots$), which can then be evaluated on a quantum processor.

4 Solve with Variational Quantum Eigensolver (VQE)

The Variational Quantum Eigensolver (VQE) is a *hybrid quantum-classical algorithm*: a quantum circuit is used to estimate energies, while a classical algorithm adjusts the circuit parameters to find the lowest energy.

Importing the Main Components We begin by importing the key building blocks of the VQE algorithm.

```python
from qiskit_algorithms import VQE
from qiskit_algorithms.optimizers import SLSQP
```

```
from qiskit.circuit.library import n_local
from qiskit.primitives import StatevectorEstimator
```

Each of these components plays a specific role:

- **VQE**: the main algorithm coordinating quantum measurements and classical optimization.
- **SLSQP**: a classical optimization algorithm that updates circuit parameters to minimize the energy.
- **n_local**: a template for building parameterized quantum circuits (ansätze).
- **StatevectorEstimator**: a simulator that computes exact expectation values, representing an ideal quantum computer.

Building the Ansatz: the Quantum Guess The ansatz is a parameterized quantum circuit that represents a *trial wavefunction*. By adjusting its parameters, we explore different possible electronic configurations of the molecule.

```
ansatz = n_local(
    num_qubits=4,
    rotation_blocks=["ry", "rz"],
    entanglement_blocks="cz",
    entanglement="linear",
    reps=3,
    insert_barriers=True
)
```

For the H_2 molecule in the STO–3G basis:

- `num_qubits = 4` matches the four spin-orbitals (bonding and antibonding, spin up and down).
- `rotation_blocks = ["ry", "rz"]` introduce adjustable angles that control the quantum state of each qubit.
- `entanglement_blocks = "cz"` add controlled-Z gates, allowing electrons to become correlated.
- `entanglement = "linear"` connects qubits in a chain (0-1, 1-2, 2-3).
- `reps = 3` repeats this pattern three times, making the ansatz expressive enough to approximate the ground state.
- `insert_barriers = True` improves circuit readability without changing its physics.

The ansatz can be viewed as a *quantum guessing machine*: it generates candidate electronic wavefunctions whose energies are evaluated.

Choosing the Classical Optimizer The optimizer decides how the circuit parameters should be updated based on measured energies:

```
optimizer = SLSQP(maxiter=100)
```

The SLSQP (Sequential Least Squares Programming) optimizer:

- is a classical, gradient-based method,
- uses up to 100 iterations to search for the minimum energy,
- plays the role of a "coach" guiding the quantum circuit toward better guesses.

The Estimator: Measuring Energy To compute energies, we use:

```
estimator = StatevectorEstimator()
```

This estimator:

- simulates an ideal quantum computer using exact statevectors,
- evaluates expectation values of the Hamiltonian efficiently,
- avoids statistical noise, making it well suited for learning and testing.

Assembling the VQE Algorithm Finally, all components are combined into a single VQE object:

```
vqe = VQE(
    estimator=estimator,
    ansatz=ansatz,
    optimizer=optimizer,
    callback=callback
)
```

At this point:

- the quantum circuit provides trial wavefunctions,
- the estimator computes their energies,
- the optimizer updates the parameters,
- the callback records the optimization history.

The result is a complete hybrid quantum-classical engine capable of approximating the ground-state energy of the H_2 molecule. VQE does not solve the Schrödinger equation directly. Instead, it repeatedly asks a quantum computer:

"How much energy does this guessed wavefunction have?"

and uses a classical optimizer to refine the guess until the lowest possible energy is found.

5 Compute the Ground-State Energy

```
result = vqe.compute_minimum_eigenvalue(qubit_op)
energy = result.eigenvalue.real + problem.nuclear_repulsion_energy
energy
```

Expected value: Approximately -1.137 Ha

6 Exercises

Exercises:

- Which part is quantum?
- Which part is classical?
- What limits today's quantum computers?
- Compare with exact solver:

```python
from qiskit_algorithms import NumPyMinimumEigensolver
exact = NumPyMinimumEigensolver()
exact_result = exact.compute_minimum_eigenvalue(qubit_op)
print(exact_result)
```

- Quantum computers don't replace classical ones; they collaborate
- Chemistry problems map naturally to qubits
- VQE is NISQ-friendly

Accuracy depends on:

- ansatz
- optimizer
- noise (on real hardware)